ShakeBox

GS805 COFFEE MACHINE

MDB Payment Guide

Android SDK (Kotlin)

Version 0.1

Date 2026-04-01

Author Anyeats World Co., Ltd.

Contact rnd@anyeats.co.kr

Classification Confidential

문서 이력

버전	날짜	내용
0.1	2026-04-01	초판 작성

목차

1. 개요

1.1 MDB 통신 구조

1.2 카드 리더 상태 머신

2. 설치 및 설정

3. API 레퍼런스

3.1 연결 관리

3.2 초기 설정

3.3 결제 명령

3.4 Event Flows

3.5 Properties

3.6 주요 클래스

4. 결제 플로우

4.1 기본 결제 플로우

4.2 결제 취소 플로우

4.3 GS805 연동: 카드결제 → 음료제조

5. 프로토콜 상세

5.1 명령어 목록

5.2 데이터 형식

6. 에러 처리

7. 기술 사양

1. 개요

본 문서는 **MDB (Multi-Drop Bus) 프로토콜** 기반 캐시리스 카드 리더 통신을 위한 **MdbCashless Android SDK**의 결제 가이드입니다. MDB-RS232 브릿지를 통해 Android 앱에서 카드리더를 제어하며, `gs805serial` 패키지에 포함되어 있습니다.

1.1 MDB 통신 구조

```
Android 앱 ←(USB RS232 9600-8N1)→ MDB-RS232 브릿지 ←(MDB bus)→ Cashless Reader
```

> **중요:** MDB는 GS805 머신과 **별도 시리얼 포트**를 사용합니다. GS805 제어와 MDB 카드리더 통신은 각각 독립된 USB 시리얼 연결이 필요합니다.

통신 규칙

- PC → 브릿지: HEX 데이터
- 브릿지 → PC: ASCII 데이터
- POLL은 브릿지가 자동 처리
- 9600 baud, 8N1

1.2 카드 리더 상태 머신

MDB 캐시리스 카드 리더는 6가지 상태를 가집니다.

#	상태	설명	전환 조건
1	**Inactive**	전원 ON/Reset 후 초기 상태	전원 ON, Reset
2	**Disabled**	Config 수신 완료	Config 응답 수신
3	**Enabled**	카드 감지 대기 중	Enable 명령 전송
4	**Session Idle**	유효 카드 감지, Vend Request 대기	유효 카드 감지
5	**Vend Requested**	VMC가 Vend Request 전송	requestVend() 호출
6	**Vending**	결제 승인, 상품 배출 중	Vend 승인 수신

```
Inactive → Disabled → Enabled → Session Idle → Vend Requested → Vending
   │                            ↑           │                                    │
   │                            └───────────┘ (세션 완료/취소)                    │
   │                            ↑                                                 │
   │                            └─────────────────────────────────────────────────┘
```

```
    │                                          (vendSuccess/vendCancel)
    └── setup() ──→ Disabled
```

2. 설치 및 설정

MdbCashless는 `gs805serial` 패키지에 포함되어 있습니다. 별도 패키지 설치 없이 동일한 Gradle 의존성을 사용합니다.

Gradle 의존성 추가 (JitPack)

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositories {
        maven("https://jitpack.io")
    }
}

// build.gradle.kts
dependencies {
    -- SDK 검증진행중, 검증 후 제공 --
    // implementation("com.github.anyeats:anyeats-***-***-***:1.0.0")
}
```

Import

```
import kr.co.anyeats.gs805serial.mdb.*
// MdbCashless, CashlessEvent, CashlessState 등 포함
```

> **참고:** MDB 클래스(`MdbCashless` , `CashlessEvent` , `CashlessState` 등)는 `kr.co.anyeats.gs805serial.mdb` 패키지에 포함되어 있습니다.

3. API 레퍼런스

3.1 연결 관리

메서드	설명
`listDevices(): List<SerialDevice>`	USB 시리얼 디바이스 목록
`suspend connect(device: SerialDevice)`	MDB-RS232 브릿지 연결
`suspend disconnect()`	연결 해제

3.2 초기 설정

메서드	설명
`suspend setup(maxPrice: Int = 0xFFFF, minPrice: Int = 0x0000)`	카드 리더 초기화
`suspend enable()`	카드 리더 활성화
`suspend disable()`	카드 리더 비활성화
`suspend cancel()`	현재 작업 취소

3.3 결제 명령

메서드	설명
`suspend requestVend(price: Int, itemNumber: Int = 1)`	결제 요청
`suspend vendSuccess(itemNumber: Int = 1)`	배출 성공 알림
`suspend vendCancel()`	벤딩 취소
`suspend cashSale(price: Int, itemNumber: Int = 1)`	현금 판매 보고
`suspend sessionComplete()`	세션 종료
`suspend requestId()`	리더 ID 요청

3.4 Event Flows

Flow	타입	설명
`eventFlow`	`SharedFlow<CashlessEvent>`	카드리더 이벤트
`connectionStateFlow`	`StateFlow<Boolean>`	연결 상태

CashlessEventType

이벤트	설명
`STATE_CHANGED`	리더 상태 변경
`CARD_DETECTED`	유효 카드 감지 (잔액 포함)
`VEND_APPROVED`	결제 승인
`VEND_DENIED`	결제 거부
`SESSION_CANCELLED`	세션 취소
`SESSION_COMPLETED`	세션 완료
`CONFIG_RECEIVED`	Config 응답 수신
`READER_ID_RECEIVED`	리더 ID 수신
`ERROR`	에러 발생
`RAW_DATA`	원시 데이터 (디버깅용)
`COMMAND_SENT`	명령 전송 로그
`ACK_RECEIVED`	ACK 수신

3.5 Properties

속성	타입	설명
`isConnected`	`Boolean`	연결 여부
`connectedDevice`	`SerialDevice?`	연결된 디바이스
`state`	`CashlessState`	현재 상태
`readerConfig`	`ReaderConfig?`	리더 설정 정보
`pendingVend`	`VendRequest?`	대기 중인 결제 요청

3.6 주요 클래스

클래스	설명
`MdbCashless`	MDB 카드리더 컨트롤러
`CashlessState`	리더 상태 enum (7종)
`CashlessEventType`	이벤트 타입 enum (12종)
`CashlessEvent`	이벤트 data class
`VendRequest`	결제 요청 data class
`ReaderConfig`	리더 설정 data class

4. 결제 플로우

4.1 기본 결제 플로우

```
val mdb = MdbCashless()

// 1. MDB 브릿지 연결 (GS805와 별도 포트)
val devices = mdb.listDevices()
mdb.connect(devices.first())

// 2. 초기 설정 (앱 시작 시 1회)
mdb.setup(maxPrice = 0xFFFF, minPrice = 0x0000)
mdb.enable()

// 3. 이벤트 수신 (Coroutine scope 내)
mdb.eventFlow.collect { event ->
    when (event.type) {
        CashlessEventType.CARD_DETECTED -> {
            // 카드 감지 → 결제 요청
            val funds = event.data["funds"] as? Int
            println("카드 감지, 잔액: $funds")
            mdb.requestVend(price = 1000, itemNumber = 1)
        }
        CashlessEventType.VEND_APPROVED -> {
            // 결제 승인 → 상품 배출
            val amount = event.data["amount"] as? Int
            println("결제 승인: $amount")
            // ... 상품 배출 로직 ...
            mdb.vendSuccess(itemNumber = 1)
            mdb.sessionComplete()
        }
        CashlessEventType.VEND_DENIED -> {
            println("결제 거부")
            mdb.sessionComplete()
        }
        else -> {}
    }
}

// 4. 종료
mdb.disconnect()
mdb.dispose()
```

4.2 결제 취소 플로우

```
// 결제 진행 중 취소
mdb.vendCancel()
mdb.sessionComplete()
```

4.3 GS805 연동: 카드결제 → 음료제조

```
// GS805 머신 + MDB 카드리더 동시 운영
val gs805 = GS805Serial(context)
val mdb = MdbCashless()

// 각각 별도 시리얼 포트에 연결
val machineDevices = gs805.listDevices()
gs805.connect(machineDevices.first())

val mdbDevices = mdb.listDevices()
mdb.connect(mdbBridgeDevice)

// MDB 초기화
mdb.setup()
mdb.enable()

// 결제 → 음료 제조 통합 플로우
launch {
    mdb.eventFlow.collect { event ->
        when (event.type) {
            CashlessEventType.CARD_DETECTED -> {
                mdb.requestVend(price = drinkPrice, itemNumber = drinkNumber)
            }
            CashlessEventType.VEND_APPROVED -> {
                gs805.makeDrink(DrinkNumber.HOT_DRINK_1)
            }
            CashlessEventType.VEND_DENIED -> {
                mdb.sessionComplete()
            }
            else -> {}
        }
    }
}

// 음료 완성 이벤트 → 결제 확정
launch {
    gs805.eventFlow.collect { event ->
        if (event.type == MachineEventType.DRINK_COMPLETE) {
            mdb.vendSuccess(itemNumber = 1)
            mdb.sessionComplete()
        }
    }
}
```

> **참고:** GS805 머신과 MDB 카드리더는 **별도의 USB 시리얼 포트**를 사용합니다. 연결 시 디바이스의 VID/PID로 구분하여 올바른 장치에 연결해야 합니다.

> **주의:** 결제 승인 후 음료 제조에 실패한 경우 반드시 `vendCancel()` → `sessionComplete()` 를 호출하여 결제를 취소해야 합니다.

5. 프로토콜 상세

5.1 명령어 목록

VMC → Card Reader (송신)

명령	HEX 데이터	설명
Config	`110001000000`	카드 리더 설정
Set Max/Min Price	`1101FFFF0000`	가격 범위 설정
Enable	`1401`	활성화
Disable	`1400`	비활성화
Cancel	`1402`	취소
Request Vend	`1300{PRICE}{ITEM}`	결제 요청
Vend Success	`1302{ITEM}`	배출 성공
Vend Cancel	`1301`	벤딩 취소
Cash Sale	`1305{PRICE}{ITEM}`	현금 판매
Session Complete	`1304`	세션 종료
Request ID	`1700`	리더 ID 요청

Card Reader → VMC (수신)

수신 데이터	설명
`00`	ACK
`01...`	Config 응답
`03{FUNDS}`	카드 감지 + 잔액
`05{AMOUNT}`	Vend 승인
`06`	Vend 거부
`07`	Session Complete ACK
`04`	세션 취소

5.2 데이터 형식

Request Vend 명령의 금액 및 상품번호는 2바이트 Big Endian HEX로 인코딩됩니다.

Request Vend 데이터 예시

금액	상품번호	HEX 데이터
10	1	`1300000A0001`
100	2	`130000640002`
500	3	`130001F40003`
1000	1	`130003E80001`

> **참고:** 금액은 10진수를 16진수로 변환하여 4자리(2바이트)로 표현합니다. 예: 1000 → 0x03E8 → `03E8`

6. 에러 처리

상황	처리 방법
명령 응답 타임아웃	100~200ms 내 응답이 없으면 재전송
카드 리더 비활성	카드 리더 전원 및 MDB 연결 상태 확인
결제 승인 후 배출 실패	반드시 `vendCancel()` 호출하여 결제 취소
세션 타임아웃	30초 내 세션 완료하지 않으면 자동 취소

```
// 에러 이벤트 처리 예시
mdb.eventFlow.collect { event ->
    if (event.type == CashlessEventType.ERROR) {
        println("MDB Error: ${event.data["error"]}")
    }

    if (event.type == CashlessEventType.VEND_DENIED) {
        // 결제 거부 시 세션 종료
        mdb.sessionComplete()
    }
}

// 음료 제조 실패 시 결제 취소
try {
    gs805.makeDrink(DrinkNumber.HOT_DRINK_1)
} catch (e: Exception) {
    // 결제 승인 상태에서 음료 제조 실패 → 반드시 취소
    mdb.vendCancel()
    mdb.sessionComplete()
}
```

> **주의:** 결제 승인(`VEND_APPROVED`) 후에는 반드시 `vendSuccess()` 또는 `vendCancel()` 을 호출해야 합니다. 호출하지 않으면 카드리더가 Vending 상태에 머물게 됩니다.

7. 기술 사양

항목	사양
프로토콜	MDB V4.2
브릿지	MDB-RS232 (Shanghai Wafer, V2021-V9.2)
통신 방식	UART RS232 (USB Serial)
전송 속도	9600 bps
데이터 형식	8N1 (8 data bits, No parity, 1 stop bit)
명령 타임아웃	200ms
세션 타임아웃	30s
테스트 리더	Nayax MDB Cashless Reader
패키지	`kr.co.anyeats.gs805serial.mdb`

© 2026 Anyeats World Co., Ltd. All rights reserved. — Confidential